

Full Stack Web Development Roadmap (1 month)

Week 1: Web Development Fundamentals

- **Day 1:** HTML5
 - **Day 2:** CSS3
 - **Day 3:** JavaScript Basics
 - **Day 4:** JavaScript Advanced
 - **Day 5:** Git & GitHub
 - **Day 6:** Mini Project – Basic Portfolio Website
 - **Day 7:** Revision and Debugging Practice
-

Week 2: React.js + API Integration

- **Day 8:** React Basics
 - **Day 9:** React Advanced
 - **Day 10:** React Advanced++
 - **Day 11:** Component Architecture and Design Patterns
 - **Day 12:** API Integration using Fetch or Axios
 - **Day 13:** Integrating React and API with Portfolio Website
 - **Day 14:** Mini Project – Notes App Frontend
-

Week 3: Backend Development with Node.js, Express.js, MongoDB

- **Day 15:** Node.js
 - **Day 16:** Express.js
 - **Day 17:** MongoDB
 - **Day 18:** Creating REST APIs with Database
 - **Day 19:** Authentication using JWT and Hashing
 - **Day 20:** Error Handling and Data Validation
 - **Day 21:** Backend Integration for Notes App
-

Week 4: Full Stack Integration, AI Integration, Deployment and Devops

- **Day 22:** MERN Stack Integration
 - **Day 23:** CORS, Proxy, Deployment Preparation
 - **Day 24:** Gemini API Implementation
 - **Day 25:** UI Polish using Libraries
 - **Day 26:** Final Project push and deployment
 - **Day 27:** Discussion, fixes and etc
 - **Day 28:** Security & Testing
-

Week 5: DevOps (optional)

- **Day 29:** DevOps Basics
 - **Day 30:** DevOps Advanced - 1
 - **Day 31:** DevOps Advanced - 2
 - **Day 32:** Kubernetes Basics & Local Deployment
 - **Day 33:** MERN App on Kubernetes + Ingress Setup
-

Full Stack Web Development Roadmap (1 month)

Week 1: Web Development Fundamentals

• Day 1: HTML5

- Semantic tags (<main>, <article>, <section>, etc.)
- Forms and attributes (required, pattern, type, autocomplete)
- Media elements (<video>, <audio>, <iframe>)
- Meta tags and SEO basics (viewport, description, robots)
- Accessibility: ARIA roles, alt, keyboard navigation, tab index
- HTML entities (©, <, etc.)

• Day 2: CSS3

- Box model, specificity, selectors
- Flexbox and Grid layout
- Media queries and responsive design
- Pseudo-elements (::before, ::after) and pseudo-classes (:hover, :nth-child)
- Transitions, animations, keyframes
- CSS variables (--primary-color)
- Responsive typography (clamp, rem, em)
- CSS reset and normalize

• Day 3: JavaScript Basics

- Data types, variables (let, const, var), operators, control flow
- Functions and scopes
- Arrays, objects, and loops (for, forEach, map, filter)
- DOM manipulation (querySelector, addEventListener)
- Events and forms handling
- JS debugging with console and breakpoints

• Day 4: JavaScript Advanced

- ES6+ features: arrow functions, destructuring, spread/rest
- Closures, hoisting, scope chain
- Promises, async/await, callback
- Modules: import, export
- LocalStorage, SessionStorage, cookies
- Debounce, throttle techniques

• Day 5: Git & GitHub

- Git basics: init, clone, add, commit, push, pull
- Branching, merging, conflicts resolution
- .gitignore, README.md, license
- Pull requests, forks, contributing to open source
- Using GitHub Pages for static deployment
- Managing Git remotes and submodules

• Day 6: Mini Project – Basic Portfolio Website

- Build a personal responsive portfolio with HTML, CSS, JS
- Sections: Home, About, Projects, Contact
- Use media queries and flexbox/grid
- Deploy using GitHub Pages
- Add favicon, loading animation, scroll-to-top button

• Day 7: Revision and Debugging Practice

- Chrome DevTools: Console, Network, Elements, Sources
 - Common JS errors and breakpoints
 - CSS debugging techniques
 - Practice using small frontend challenges
-

Week 2: React.js + API Integration

• Day 8: React Basics

- create-react-app setup, file cleanup
- JSX syntax, components
- Props and state (useState)
- Handling events and rendering lists
- Component folder structure

• Day 9: React Advanced

- useEffect and side effects
- Conditional rendering (&&, ternary)
- Forms: controlled vs uncontrolled components
- State lifting

• Day 10: React Advanced++

- useRef, useContext, and custom hooks
- Error boundaries
- Lazy loading and code splitting (React.lazy, Suspense)
- Fragments and key props
- Performance optimization (React.memo, useMemo, useCallback)

• Day 11: Component Architecture and Design Patterns

- Smart vs dumb components
- Higher-order components (HOC)
- Context API for global state
- Reusable UI components (buttons, inputs, cards)
- Folder and file organization in scalable projects

• Day 12: API Integration using Fetch or Axios

- REST APIs with Fetch/Axios
- Async data handling with loading/error states
- API call abstraction using services
- Displaying dynamic data in UI
- Error handling with try/catch + toast notifications

• Day 13: Integrating React and API with Portfolio Website

- Add project data dynamically from API
- Add contact form with form submission logic
- Toast notifications and success/error handling
- Basic pagination or filtering
- Use react-router-dom to enable routes

• Day 14: Mini Project – Notes App Frontend

- Notes listing and creation using state
 - Search and filter functionality
 - LocalStorage integration
 - Clean UI with components, props, animations and transitions
-

Week 3: Backend Development with Node.js, Express.js, MongoDB

• Day 15: Node.js

- Node.js architecture and event loop
- Built-in modules (fs, path, os, url)
- Creating an HTTP server
- package.json, npm scripts, nodemon, dotenv
- import/require module systems
- Intro to npm, npx, and version control

• Day 16: Express.js

- Express app setup
- Routing (app.get, app.post, app.delete, app.put)
- Middleware and request handling
- Using body-parser, morgan, cors
- Creating modular route/controller files
- Route parameters and query strings

• Day 17: MongoDB

- MongoDB Atlas & Compass setup
- Mongoose schemas and models
- CRUD operations with Mongoose
- Data relationships with ref and populate()
- Query filters and projections

• Day 18: Creating REST APIs with Database

- REST principles and route structure
- Create CRUD endpoints (Notes API)
- Postman testing
- Use .env for DB connection
- Status codes and error handling
- Use express-async-handler to simplify try/catch

• Day 19: Authentication using JWT and Hashing

- User model with password hashing (bcrypt)
- Register/login routes
- JWT generation and verification (jsonwebtoken)
- Protecting private routes (auth middleware)
- Storing tokens securely (HTTP headers, cookies)

• Day 20: Error Handling and Data Validation

- Express error-handling middleware
- Validating requests using Joi
- Centralized error response structure
- Handling async errors with try/catch
- CORS configuration

• Day 21: Backend Integration for Notes App

- Connect frontend notes app to backend API
 - Use JWT for authenticated user notes
 - Fetch, create, delete, and update notes via API
 - Display server-side validation errors
-

Week 4: Full Stack Integration, AI Integration, Deployment

• Day 22: MERN Stack Integration

- Connect React frontend with Express backend
- Handle auth tokens (JWT) with Axios interceptors
- Use proxy for local development
- Conditional rendering based on auth state

• Day 23: CORS, Proxy, Deployment Preparation

- Enable CORS for API access
- Configure frontend and backend .env files
- Build React app for production (npm run build)
- Setup backend for deployment (Vercel , render , heroku)

• Day 24: Gemini API Implementation

- Set up Gemini API via Google AI Studio
- Generate access key and secure it
- Create a chatbot or summarizer feature
- Backend integration using fetch or axios
- Use prompt engineering for better responses
- Rate limits, token usage, and error handling

• Day 25: UI Polish using Libraries

- Install and configure UI libraries (MUI, Shadcn, Tailwind)
- Use reusable components from ui libraries (modals, buttons, cards)
- Add data charts using Chart.js or Recharts
- Responsive design improvements
- Add dark mode toggle
- Animate with Framer Motion
- Lazy loading, image optimization

• Day 26: Final Project push & Deployment

- Write documentation in README.md
- Add environment setup guide ,preview screenshots and video, .env.example and LICENSE
- Deploy frontend (Vercel) and backend (Render)
- Add environment variables in deployment platform
- Google indexing and SEO basics

• Day 27: Discussion & Deployment

- Code review, refactoring, feedbacks
- Fix merge conflicts, improve commit history
- Debug and polish UI/UX
- Bug hunting , raising issues and fixing

• Day 28: Security & Testing

- Helmet.js , rate limit , validation
 - Unit testing with jest
 - Integration & auth testing
-

Week 5: Devops

• Day 29: DevOps Basics

- What is DevOps: CI/CD, automation, testing, containerization
- GitHub Actions: auto deployment on push & workflows
- Build CI/CD pipeline with GitHub Actions pipeline
- Intro to Docker: image, container, Dockerfile
- Dockerize MERN app (Dockerfile, docker-compose) (basic)

• Day 30: DevOps Advanced - 1

- Setup environment variables for prod
- Dockerize MERN app (Dockerfile, docker-compose) (advanced)
- Monitoring and scaling considerations
- Version tagging, rollback strategies

• Day 31: DevOps Advanced - 2

- Use of reverse proxy like Nginx
- Deploying in cloud platforms: AWS, GCP, Railway, Fly.io
- Domain linking

• Day 32: Kubernetes Basics & Local Deployment

- pods, deployments, services, nodes
- Install and configure Minikube + kubectl CLI
- Create basic deployment and service YAML
- Deploy a basic static website (like Nginx or Node.js)
- View your website using a local Kubernetes URL

• Day 33: MERN App on Kubernetes + Ingress Setup

- Write deployments and services for frontend and backend
 - Connect backend with MongoDB Atlas using env variables
 - Install Ingress controller (traffic router) using Minikube addon
 - Create Ingress YAML to link our site to a local or custom domain
 - Map local domain using /etc/hosts (e.g., notes.local)
 - Access full MERN app through browser with Kubernetes routing
-